Date: 22-10-2026

CO-2 - AT-1

ANALYTICAL PROBLEM SOLVING

COURES CODE: DSA0308

COUSRE NAME: NLP

Reg no: 192472313 **Name: Pavan Jana**

1. Morphological Analysis for Document Indexing

Aim

To decompose the words connected, connecting, and connection into their root and suffix components and normalize them for document retrieval.

Method and analysis

- Root word: connect
- Inflectional suffixes: -ed, -ing indicate grammatical forms.
- Derivational suffix: -ion forms the noun connection from the verb connect.
- Normalization: Map all three words to the common base connect for indexing.

Input word	Root	Suffix	Classification	Normalized form
connected	connect	-ed	Inflectional	connect
connecting	connect	-ing	Inflectional	connect
connection	connect	-ion	Derivational	connect

Python program:

```
words = ["connected", "connecting", "connection"]

rules = {

    "connected": ("connect", "-ed", "Inflectional"),

    "connecting": ("connect", "-ing", "Inflectional"),

    "connection": ("connect", "-ion", "Derivational")

}

print(f"{'Word':<15}{'Root':<12}{'Suffix':<10}{'Type':<15}{'Normalized'}")

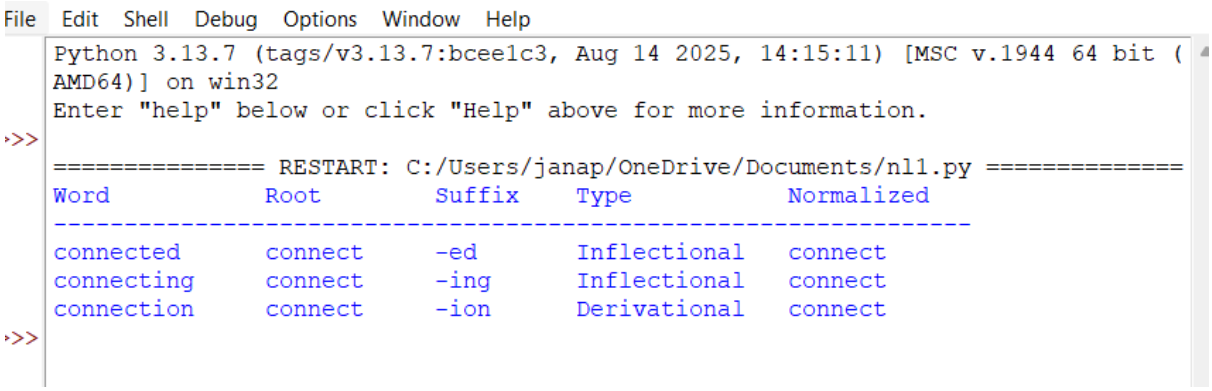
print("-" * 65)

for word in words:

    root, suffix, kind = rules[word]

    print(f"{word:<15}{root:<12}{suffix:<10}{kind:<15}{root}")
```

Output:



Interpretation: The three words can be indexed under connect, allowing a search for one form to retrieve documents containing the other forms. The program uses predefined morphological rules for the given dataset.

2. Morphological Parsing for Sentiment Analysis

Aim

To identify prefixes, suffixes, and base forms in unhappy, happiness, and happily, while classifying the transformations.

Method and analysis

- Base word: happy
- Prefix: un- adds a negative meaning.
- Suffixes: -ness forms a noun, while -ly forms an adverb.
- Classification: These are derivational transformations because they create words with new meanings or grammatical categories.

Input word	Prefix	Suffix	Base form	Classification	Normalized root
unhappy	un-	None	happy	Derivational	happy
happiness	None	-ness	happy	Derivational	happy
happily	None	-ly	happy	Derivational	happy

Note: happiness and happily involve a spelling change from y to i. The parser handles this by mapping both words to happy.

Python program

```
words = ["unhappy", "happiness", "happily"]

rules = {

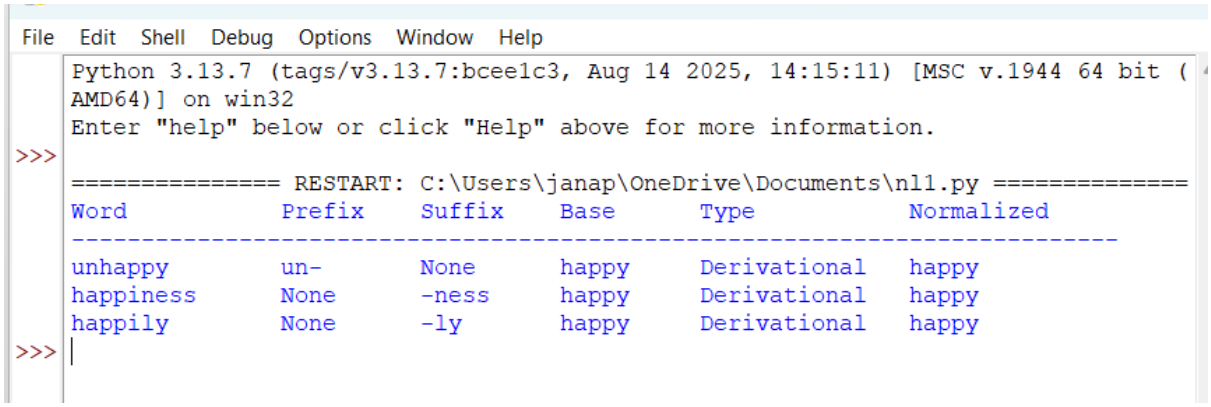
    "unhappy": ("un-", "None", "happy", "Derivational"),

    "happiness": ("None", "-ness", "happy", "Derivational"),

    "happily": ("None", "-ly", "happy", "Derivational")
```

```
}
print(f"{'Word':<15}{'Prefix':<10}{'Suffix':<10}"
      f"{'Base':<10}{'Type':<15}{'Normalized'}")
print("-" * 75)
for word in words:
    prefix, suffix, base, kind = rules[word]
    print(f"{word:<15}{prefix:<10}{suffix:<10}"
          f"{base:<10}{kind:<15}{base}")
```

Output:



Interpretation: Normalizing the words to happy groups related words for sentiment analysis. However, unhappy expresses negative sentiment, so a sentiment analysis system should preserve the prefix or original word to avoid losing polarity information.

3. Stemming-Based Preprocessing for a Search Engine

Aim

To remove suffixes from played, player, and playing, identify their stems, and generate normalized forms for search indexing.

Method and analysis

- Inflectional suffixes: -ed and -ing represent grammatical variations of the verb.
- Derivational suffix: -er forms the noun player, meaning a person who plays.
- Common stem: play

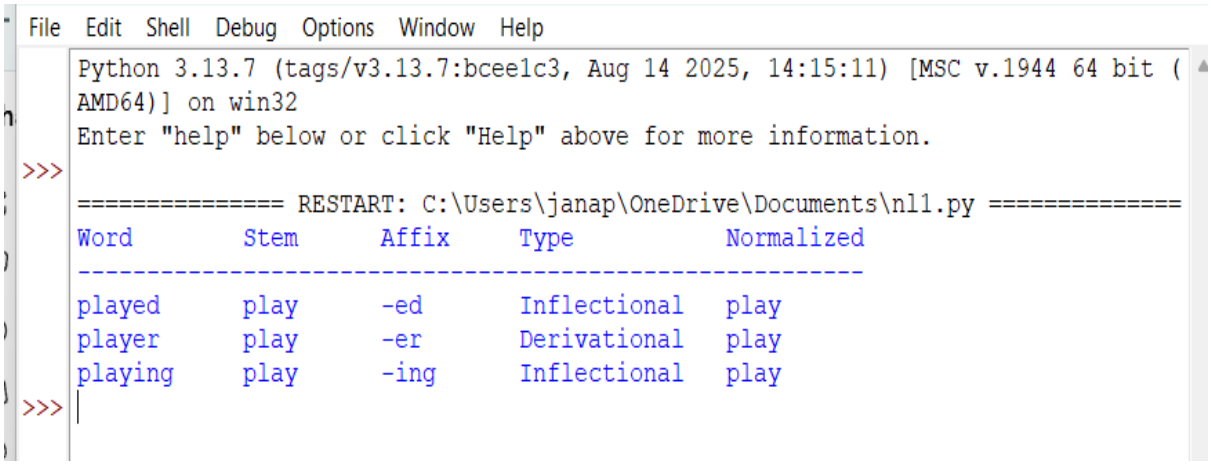
Original word	Extracted stem	Removed affix	Transformation type	Normalized form
played	play	-ed	Inflectional	play
player	play	-er	Derivational	play
playing	play	-ing	Inflectional	play

Python program:

```
words = ["played", "player", "playing"]
rules = {
    "played": ("play", "-ed", "Inflectional"),
    "player": ("play", "-er", "Derivational"),
    "playing": ("play", "-ing", "Inflectional")
}
print(f"{'Word':<12}{'Stem':<10}{'Affix':<10}"
      f"{'Type':<15}{'Normalized'}")
print("-" * 57)
```

```
for word in words:
    stem, affix, kind = rules[word]
    print(f"{word:<12}{stem:<10}{affix:<10}"
          f"{kind:<15}{stem}")
```

Output:



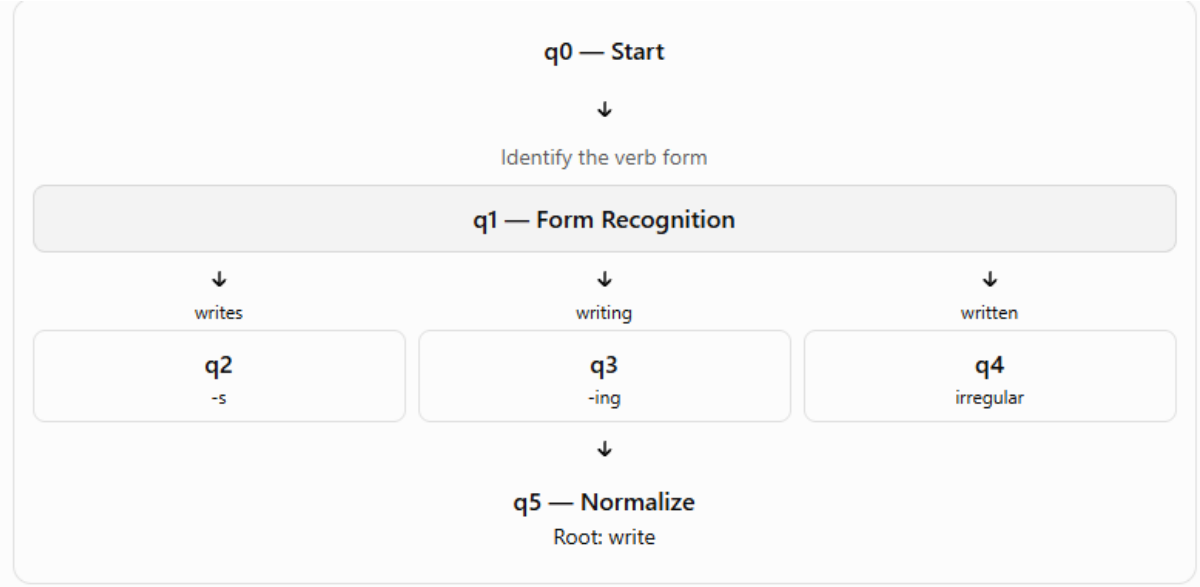
Interpretation: The search engine indexes all three words under play. This improves retrieval when a user searches for a word but documents contain a different form. The player form is derivational, so a practical search system may retain the original word to distinguish people who play from the action of playing.

4. Finite-State Morphological Parsing

Aim

To build a finite-state parser for writes, writing, and written, handling regular and irregular verb forms and mapping them to the root write.

Method and state transitions



The writes and writing forms follow regular inflectional patterns. Written is an irregular past participle, so it requires a special mapping to write.

Word	Morphological breakdown	Classification	State path	Normalized form
writes	write + -s	Regular inflection	q0 → q1 → q2 → q5	write
writing	write + -ing	Regular inflection	q0 → q1 → q3 → q5	write
written	writ + -ten; irregular mapping to write	Irregular inflection	q0 → q1 → q4 → q5	write

Python program:

```
words = ["writes", "writing", "written"]

transitions = {

    "writes": ("q2", "write + -s", "Regular inflection"),

    "writing": ("q3", "write + -ing", "Regular inflection"),

    "written": ("q4", "writ + -ten", "Irregular inflection")

}

print(f"{'Word':<12}{'State Path':<25}"

      f"{'Breakdown':<22}{'Type':<22}{'Root'}")

print("-" * 100)

for word in words:

    state, breakdown, kind = transitions[word]

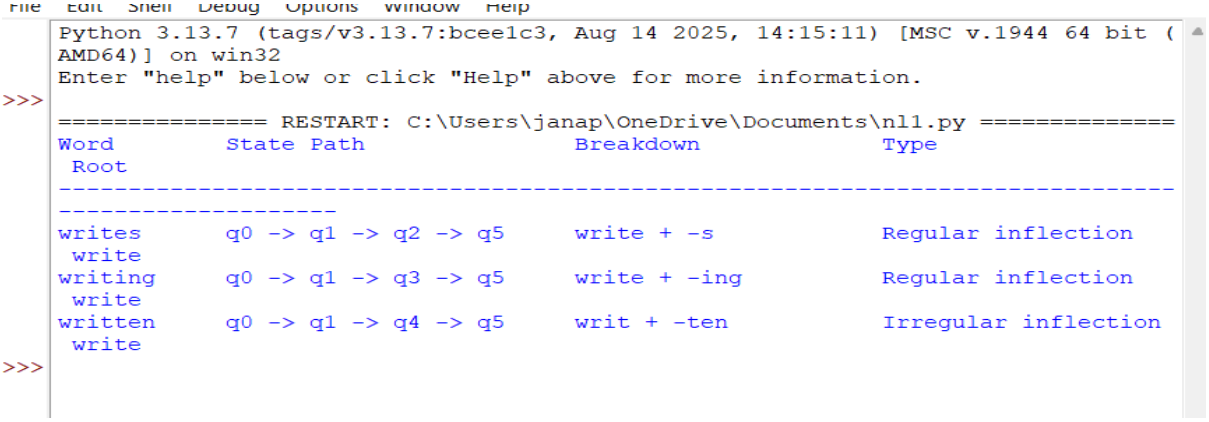
    path = f"q0 -> q1 -> {state} -> q5"

    root = "write"

    print(f"{word:<12}{path:<25}{breakdown:<22}"

          f"{kind:<22}{root}")
```

Output:



5. Porter Stemmer-Based Preprocessing for Document Retrieval

Aim

To apply Porter stemming rules to relational, relation, and relate, showing the stemming steps and final normalized form.

Method and analysis

The Porter stemming algorithm removes suffixes in a sequence of steps. For these words, the relevant rules are:

1. Step 2: Replace -ational or -ation with -ate when the rule's conditions are satisfied.
2. Step 5a: Remove the final e when the Porter measure condition is satisfied.

Original word	Step 2 result	Step 5a result	Final stem
relational	relate	relat	relat
relation	relate	relat	relat
relate	Not applicable	relat	relat

Step-by-step explanation

- relational → replace -ational with -ate → relate → remove final e → relat.
- relation → replace -ation with -ate → relate → remove final e → relat.
- relate → remove final e under Step 5a → relat.

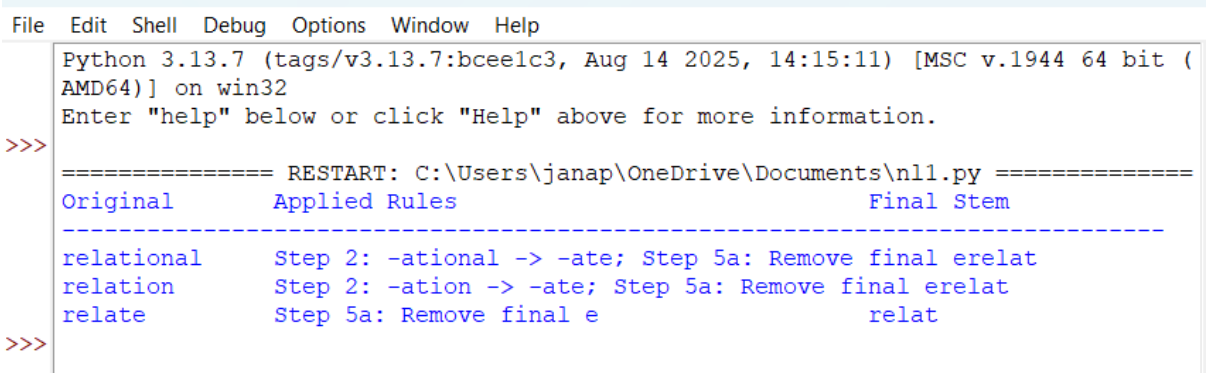
Python program:

```
words = ["relational", "relation", "relate"]

def porter_stem(word):
    steps = []
    if word.endswith("ational"):
        word = word[:-7] + "ate"
```

```
        steps.append("Step 2: -ational -> -ate")
    elif word.endswith("ation"):
        word = word[:-5] + "ate"
        steps.append("Step 2: -ation -> -ate")
    if word.endswith("e") and len(word) > 4:
        word = word[:-1]
        steps.append("Step 5a: Remove final e")
    return steps, word
print(f"{'Original':<15}{'Applied Rules':<42}{'Final Stem'}")
print("-" * 78)
for original in words:
    steps, stem = porter_stem(original)
    rule_text = "; ".join(steps) if steps else "No rule applied"
    print(f"{original:<15}{rule_text:<42}{stem}")
```

Output:



Interpretation: All three words are reduced to relat, allowing a document retrieval system to index them under the same stem. relat is a stem, not necessarily a valid standalone English word.